



L OVELY
P ROFESSIONAL
U NIVERSITY

CSB205: SYSTEM DESIGN

Minor Project

Topic: Cloud-Based Disaster Recovery Automation System (CDRAS)

Name: Perineni Lakshith

Reg No: 12401218

Section: K24BX

Cloud-Based Disaster Recovery Automation System (CDRAS)

1. Requirements Pack

1.1 Stakeholder Analysis & Prioritization

The following table summarizes the primary stakeholders and their vested interests in the successful implementation of CDRAS.

Stakeholder Role	Priority	Key Interests
Cloud Administrator	High	System reliability, operational automation.
IT Manager	High	Fast Recovery Time Objective (RTO), regulatory compliance.
Application Developer	Medium	Continuous application uptime during incidents.
End User	Medium	Consistent availability of business applications.
Compliance Officer	Low	Legal requirements, comprehensive audit and recovery logs.

1.2 Functional Requirements

The system must perform the following core functions:

- **Monitoring:** Real-time system monitoring to detect infrastructure failures.
- **Detection:** Automatic detection of server or storage outages.
- **Backup & Replication:** Cloud-based data backup and replication across designated regions.
- **Orchestration:** Ability to trigger automated recovery workflows (runbooks/Lambda

functions).

- **Failover:** Automatic, immediate failover to a pre-configured secondary region upon failure detection.
- **Notification:** Proactive notification of administrators regarding incidents and recovery progress.
- **Logging:** Maintenance of immutable recovery logs suitable for compliance audits.

1.3 Non-Functional Requirements (NFRs)

The quality and performance targets for CDRAS are defined as follows:

- **Availability:** Target uptime of **99.99%** (less than 52.6 minutes of downtime per year).
- **Durability:** Data backup durability target of **99.999999999% (11 9's)**.
- **Latency:** Monitoring updates must be processed in **<200 ms**.
- **Security:** Mandate end-to-end data encryption and strict Identity and Access Management (IAM) policies.
- **Scalability:** The system must be able to seamlessly scale horizontally with the increasing number of monitored systems.

1.4 Constraints & Assumptions

The implementation is bound by the following non-negotiable limitations:

- **Connectivity:** Continuous Internet connectivity is required for all cloud operations and communications.
 - **Infrastructure:** The solution must be implemented exclusively on **AWS or Azure** cloud infrastructure.
 - **Regions:** Recovery regions must be preconfigured and validated prior to system deployment.
 - **Testing:** Initial free-tier testing is limited by a maximum backup size of **≤ 5 TB**
-

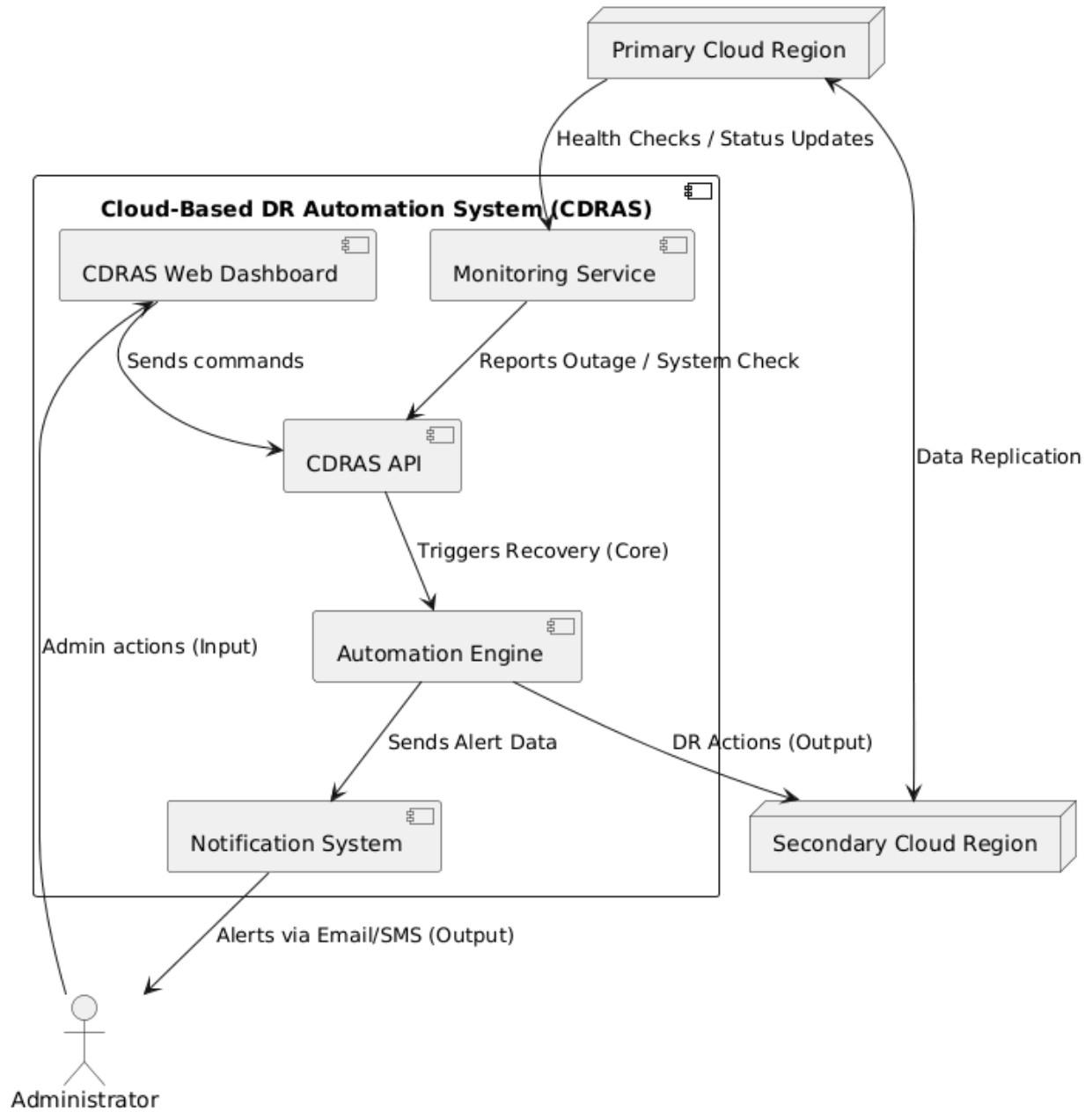
2. Diagrams & Conceptual Architecture

2.1 System Context Diagram

CDRAS operates as a central orchestration layer between the administrators and the infrastructure.

- **Input:** Admin actions (via Web Dashboard) and system health checks (from Primary Cloud).
- **Core:** Monitoring Service, Automation Engine, and CDRAS API.
- **Output:** Disaster Recovery actions in the Secondary Cloud Region and alerts via the Notification System.

System Context Diagram (CDRAS)

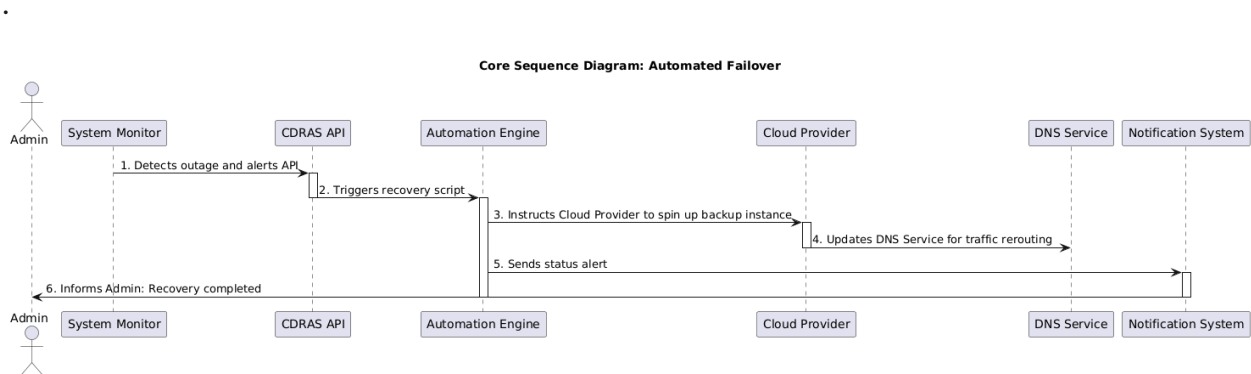


2.2 Use Cases & User Stories

Role	Use Case/Scenario (Goal)	Why (Benefit)
Admin/Operations	"As an Admin , I want the system to send high-priority alerts if a primary service fails, even during automated failover , so I can verify the success of the automated recovery and step in if necessary."	Ensures human verification and oversight during critical moments, supporting the RTO.
Developer/SRE	"As a Developer/SRE , I want to be able to schedule and run non-disruptive recovery drills automatically every week , so I can continuously test and ensure the DR readiness of the application code and IaC runbook."	Validates RTO/RPO compliance and ensures the DR runbook doesn't drift.
Auditor/Compliance	"As an Auditor , I want to access immutable logs of every recovery action taken by the system (start time, end time, status), so I can meet regulatory and compliance reporting requirements."	Provides required audit trail and proves RTO/RPO targets are met consistently.

2.3 Core Sequence Diagram (Failover Example)

- 1. **System Monitor** detects outage and alerts **CDRAS API**.
- 2. **CDRAS API** triggers the **Automation Engine** recovery script.
- 3. **Automation Engine** instructs the **Cloud Provider** to spin up a backup instance in the secondary region.
- 4. **Cloud Provider** updates the **DNS Service** for traffic rerouting.
- 5. **Automation Engine** sends a status alert to the **Notification System**.
- 6. **Notification System** informs the **Admin** that the recovery is completed

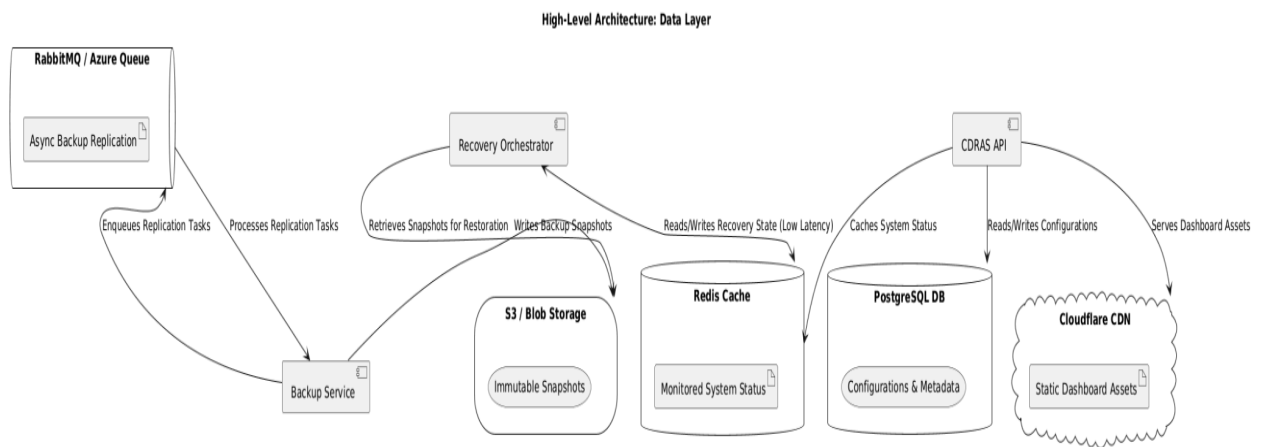


2.4 High-Level Architecture: Service Layer

Component	Technology	Responsibility
Frontend	React / Flask	Web Dashboard for administration and reporting.
Backend	Python Flask / FastAPI	Core business logic, monitoring, backup, and notification processing.
Orchestrator	Serverless/Runbook	Handles stateful recovery workflows and automated failover scripts.

2.5 High-Level Architecture: Data Layer

- **Data Stores:** PostgreSQL for configurations and user data; S3/Blob Storage for immutable snapshot and backup storage.
- **Queues:** RabbitMQ or Azure Queue for asynchronous task processing (e.g., long-running backup replication).
- **Cache:** Redis utilized to store recently monitored system status for low-latency dashboard viewing.
- **CDN:** Cloudflare (or equivalent) for highly performant delivery of static dashboard assets.



3. Engineering Notes

3.1 Capacity Planning & Sizing

- **Monitoring Load:** 100 systems × 10 checks/min = **1000 API calls/min** at peak.
- **Storage Load:** 1 GB avg backup/system/day = **100 GB/day** total storage consumption.
- **Recovery Time Objective (RTO):** Target < **5 minutes**.
- **Recovery Point Objective (RPO):** Target < **1 minute** data loss.

3.2 API Specification & Data Model

Sample API Specification (Recovery Endpoint):

POST /api/v1/recover

BODY: {

 "region": "east-us",

 "system_id": "srv-101"

}

RESPONSE: {

 "status": "recovered",

 "time": "2025-10-30T12:00:00Z"

}

Core Data Model Schemas:

- System(system_id, name, region, status)
- Backup(backup_id, system_id, timestamp, storage_path)
- RecoveryLog(log_id, system_id, event, timestamp)
- User(user_id, role, email)

3.3 Consistency Choices & Caching

- **Strong Consistency:** Applied to critical states (e.g., recovery status, backup confirmations).
- **Eventual Consistency:** Applied to non-critical data (e.g., reporting dashboards, historical metrics).
- **Caching:** Redis is mandatory for low-latency retrieval of monitored system status.
- **Indexing:** Mandatory indexing on key lookup fields: system_id and timestamp.

3.4 Resiliency Mechanisms

- **Retries:** 3 attempts maximum with exponential backoff for external cloud API calls.
- **Timeouts:** Critical cloud API calls are enforced with a strict **5-second timeout**.
- **Circuit Breaker:** Logic is implemented to open the circuit after **3 consecutive failures**, preventing cascading downstream overload.

3.5 Rate Limiting Policy

Scope	Limit
User-Based Limits	100 monitoring requests/min per individual user.
System-Wide Limits	5000 API calls/min maximum system-wide throughput.

3.6 Observability Stack

- **Logs:** Stored in the **ELK** (Elastic Stack) for centralized search and analysis.
- **Metrics:** Monitored via **Prometheus + Grafana** (tracking RTO, RPO, CPU, backup frequency).
- **Traces:** Implemented using **OpenTelemetry** for distributed tracing across all microservices.

3.7 Maintenance & DR Drill Plan

- **Routine Maintenance:** Weekly backup integrity verification; automated patch deployment schedule.
- **Disaster Recovery Drills:** Mandatory monthly automated failover drills (to test RTO/RPO compliance) and quarterly comprehensive scalability tests.

4. Quality Targets & Trade-Offs

4.1 Service Level Objectives (SLOs)

Metric	Target
Availability	≥ 99.99%
Recovery Time Objective (RTO)	< 5 minutes
Recovery Point Objective (RPO)	< 1 minute
Response Latency	< 200 ms (Monitoring Updates)

4.2 Scalability Plan

To ensure horizontal scaling and performance:

- **Scaling Model:** Utilize **Kubernetes or Azure AKS** for horizontal auto-scaling of stateless services.
- **Partitioning:** Backup data storage will be partitioned (sharded) by region or customer identifier.
- **Content Delivery:** Use a CDN for highly performant delivery of static dashboard assets.

4.3 Trade-Off Discussion

The following architectural trade-offs were consciously accepted:

Decision	Benefit	Drawback
Multi-region Backup	Higher reliability and resilience against regional failure.	Results in higher operating costs.
Eventual Consistency	Better horizontal scalability and faster writes.	Dashboards may occasionally display stale data.
Serverless Recovery	Fast auto-scaling and low operational overhead.	Potential for initial cold start delays during recovery launch.

5. Conclusion

The Cloud-Based DR Automation System (CDRAS) is designed to transition disaster recovery from a costly, manual process to a resilient, **highly automated cloud native function**. By adhering to strict **RTO (<5 min) and RPO (<1 min) objectives**, the architecture prioritizes speed and data integrity through its dedicated orchestration and robust observability stack. The strategic use of queuing and caching ensures both **scalability and consistent uptime**, setting a high standard for business continuity.